Documentation

(Longest Subarray with Equal 0s and 1s)

First algorithm:

(Non-Recursive / Iterative)

iterative Prefix Sum Algorithm:

1. Pseudo code :

```
FUNCTION checkArray(array, size):
    max ← 0                                          →1
    FOR x FROM 0 TO size-1:                          →n
        IF array[x] == 0:
            max ← x + 1                              →1*n
        FOR y FROM 0 TO x-1:                         →n(n-1)/2
            IF array[x] == array[y]:
                IF (x - y) > max:
                    max ← x - y
                END IF
            END IF
        END FOR
    END FOR
    RETURN max                                       →1
END FUNCTION
```

```
FUNCTION LongestSubarray(arr, size):
    length ← 0                                          →1
    array ← allocate memory of size integers            →1
    FOR i FROM 0 TO size-1:                             →n
        IF arr[i] == 0:
            length ← length - 1
        ELSE:
            length ← length + 1
        array[i] ← length
    END FOR
    result ← checkArray(array, size)                    →T1(check array)
    free memory of array                                →1
    RETURN result                                       →1
END FUNCTION
```

2. Analysis & complexity (steps):

$T1(n) = 1 + n + n + n + n(n-1)/2 + n(n-1)/2 + n(n-1)/2 + n(n-1)/2 + 1$

$= 2 + 3n + 4 \times n(n-1)/2 = 2 + 3n + 2n(n-1)$

$= 2 + 3n + 2n^2 - 2n = 2n^2 + n + 2$

$T(n) = \text{LongestSubarray} + \text{checkArray}$

$= (3n + 4) + T1(n)$

$= (3n + 4) + (2n^2 + n + 2)$

$= 2n^2 + 4n + 6$

Since the highest power of n is 2,the growth rate is Polynomial

Time Complexity	$O(n^2)$
Space Complexity	O(n)

Second algorithm: (recursive)

Recursive Prefix Sum Algorithm:

1. Pseudo code :

```
FUNCTION solveRecursive(sumArray, size, index):


   // Base Case
   IF index >= size:
      RETURN 0
   END IF


   currentMax ← 0


   // Check if prefix sum = 0 (balanced from start)
   IF sumArray[index] == 0:
      currentMax ← index + 1
   END IF


   // Check all previous positions
   FOR j FROM 0 TO index-1:
```

```
    IF sumArray[index] == sumArray[j]:
        IF (index - j) > currentMax:
            currentMax ← index - j
        END IF
    END IF
END FOR

// Recursive call for next index
nextMax ← solveRecursive(sumArray, size, index + 1)

// Return the larger of current and next
IF currentMax > nextMax:
    RETURN currentMax
ELSE:
    RETURN nextMax
END IF
```

```
END FUNCTION


FUNCTION LongestSubarray(arr, size):
    length ← 0                                          →1
    array ← allocate memory of size integers            →1
    FOR i FROM 0 TO size-1:                             →n
        IF arr[i] == 0:
            length ← length - 1
        ELSE:
            length ← length + 1
        array[i] ← length
    END FOR
    result ← solveRecursive (array, size,0)
                                                →T1(solveRecursive)
    free memory of array                        →1
    RETURN result                               →1
END FUNCTION
```

2. Analysis & complexity (steps):

Recursive call and loop :

$$T(n) = T(n-1) + 3n + 4$$

Base case :

$$T(0) = 1$$

$$T1(n) = T(n-1) + 3n + 4$$

$$= T(n-2) + 3(n-1) + 4 + 3n + 4$$

$$= T(n-2) + 3(2n-1) + 8$$

$$= T(n-3) + 3(n-2) + 4 + 3(2n-1)+8$$

...

$$=T(0) + 3[1+2+...+n] + 4n$$

$$= 1 + 3\times n(n+1)/2 + 4n$$

$$= (3/2)n^2 + (5/2)n + 1$$

$T(n) = \text{LongestSubarray} + \text{solveRecursive}$

$= (3n + 4) + T1(n)$

$= (3n + 4) + ((3/2)n^2 + (5/2)n + 1$

$= (3/2)n^2 + (11/2)n + 5$

Since the highest power of n is 2,the growth rate is Polynomial

Time Complexity	$O(n^2)$
Space Complexity	$O(n)$

Comparison

	Non-Recursive	Recursive
Time Analysis	$2n^2 + 4n + 6$	$(3/2)n^2 + (11/2)n + 5$
Time Complexity	$O(n^2)$	$O(n^2)$
space Complexity	Array space :O(n) Stack space :O(1) Total :O(n)	Array space :O(n) Stack space :O(n) Total :O(n)
performance	Faster by 3n	Slower by 3n
implementation	Easy : loops	Complex : function call and base case
stability	stable	stable
execution type	Iterative	Linear Recursion
description	Loop through all pairs iteratively	Process one index per call recursively